Movie Theater Ticketing Software System (MTTS)

Software Requirements Specification

Version 1.5

04-09-2026

Group #6
Eric Daniel Alfonso, Leland Cuellar, Matthew Aquino

Prepared for
CS 250- Introduction to Software Systems
Instructor: Gus Hanna, Ph.D.
Fall 2023

Revision History

Date	Description	Author	Comments
2/05/26	Version 1.0	Leland, Eric, Matthew	First Revision – Section 1
2/12/26	Version 1.1	Leland, Eric, Matthew	Second Revision – Section 2
2/17/26	Version 1.2	Leland, Eric, Matthew	Third Revision - Section 3
2/19/26	Version 1.3	Leland, Eric, Matthew	Fourth Revision, Section 1-3
2/26/26	Version 1.4	Leland, Eric, Matthew	Sections 3-4
4/8/26	Version 1.5	Leland, Eric, Matthew	Section 5 – Data management

Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

Signature	Printed Name	Title	Date
	Leland, Eric, Matthew	Software Eng.	
	Dr. Gus Hanna	Instructor, CS 250	

Table of Contents

Revision History ii

Document Approval ii

1. Introduction 1

1.1 Purpose 1
1.2 Scope 1
1.3 Definitions, Acronyms, and Abbreviations 1
1.4 References 1
1.5 Overview 1

2. General Description 2

2.1 Product Perspective 2
2.2 Product Functions 2
2.3 User Characteristics 2
2.4 General Constraints 2
2.5 Assumptions and Dependencies 2

3. Specific Requirements 2

3.1 External Interface Requirements 3
3.1.1 User Interfaces *3*
3.1.2 Hardware Interfaces *3*
3.1.3 Software Interfaces *3*
3.1.4 Communications Interfaces *3*
3.2 Functional Requirements 3
3.2.1 <Functional Requirement or Feature #1> *3*
3.2.2 <Functional Requirement or Feature #2> *3*
3.3 Use Cases 3
3.3.1 Use Case #1 ..... *3*
3.3.2 Use Case #2 ..... *3*
3.4 Classes / Objects 3
3.4.1 <Class / Object #1> *3*
3.4.2 <Class / Object #2> *3*
3.5 Non-Functional Requirements 4
3.5.1 Performance *4*
3.5.2 Reliability *4*
3.5.3 Availability *4*
3.5.4 Security *4*
3.5.5 Maintainability *4*
3.5.6 Portability *4*
3.6 Customer Architectural Diagram 4
3.7 Admin Architectural Diagram 4
3.8 UML Class Diagram 4

4. Development Plan 4

4.1 Development Timeline 5
4.2 Github Links 5

5. Testing the System 5

5.1 Test Plan & Description of Cases 5

6. Data Management Strategy 5

6.1 Database System Plan 5

6.2 SQL Tables Architecture Diagram & Description 5

A. Appendices **5**
A.1 Appendix 1 5
A.2 Appendix 2 5

1. Introduction

The introduction of this SRS provides an overview of the document. It includes its purpose, scope, definitions, acronyms, and abbreviations, and references. The goal of this SRS is to provide a detailed and comprehensive guide to the software of the **Movie Theater Ticketing Software System.** This document also focuses on the functionality required by the **Fun Films Movie Theater Co.** while defining high-level product features.

1.1 Purpose

The purpose of this document is to compile and analyze all of the ideas related to the development of the **Movie Theater Ticketing Software System**. This document goes over the details of the ideas and development that went into this ticketing system. We will also attempt to describe how we see this product being used, go over developmental plans being considered, and describe the plans for future operation of the system.

Overall, this document contains the goals and specifications of the Move Theater Ticketing Software System. Inside there will be descriptions of the specifications for software, hardware and functionality. This document is intended for everyone involved in the design and construction of this software system, including but not limited to: the developers, the design coordinators at Fun Film Movie Theater Co., and even the end users or system administrators of Fun Film Movie Theater Co.

1.2 Scope

The customer of the **Movie Theater Ticketing Software System** is the **Fun Films Movie Theater Co.**, and the end users are customers of that company and the System Administrators at that company.
This **Movie Theater Ticketing Software System** will provide an intuitive and simple ticketing system that lets users purchase tickets for specific movie screenings online at all of the Fun Films Movie Theater Co. locations in San Diego. Theater Company administrators will be able to add/set: Theater buildings, theaters within the buildings, available seats / ticket availability, movies and their time slots, loyalty/rewards system, and the refund policy for tickets.
In addition, it will allow for easy verification of purchase by theater employees through QR codes sent through text and email, provided by the system to users.
This **Movie Theater Ticketing Software System** will NOT provide users the ability to resell tickets or buy concessions in advance.
It will not provide the ability to look up / buy transportation to the movie theaters.

1.3 Definitions, Acronyms, and Abbreviations

MTTSS	Movie Theater Ticketing Software System
FFMTC	Fun Films Movie Theaters Co.
Admin	Theater System Administrators in charge of operating the MTTSS
Web app	Web-application
UI	User Interface
UX	User Experience
TC	Touch Computer

This subsection should provide the definitions of all terms, acronyms, and abbreviations required to properly interpret the SRS. This information may be provided by reference to one or more appendixes in the SRS or by reference to other documents.

1.4 References

The References are:

1. IEEE Std 830-1998 (Institute of Electrical and Electronics Engineers, Inc.)

1.5 Overview

The rest of the SRS is as follows: General Description of the project, Specification Requirements, Analysis Model, Change Management Process, and Appendices. Section #2 will go over product perspectives and functions, user constraints, general constraints, and assumptions/dependencies. Section #3 will discuss use cases, design constraints, and many different types of requirements. Section #4 will show sequence, data flow, and state transition diagrams.

2. General Description

2.1 Product Perspective

This MTTSS is a web based application designed to be used by FFMTC so their clients can buy movie tickets online. This MTTSS is not going to be embedded as a smaller part of any larger system. It will however, be connected to other large systems, like banking and credit card systems. This system consists of a web app, backend API server, and a database.

a) System Interfaces

i) The system will interface with Stripe's API to process payments reliably and securely.
ii) The system will also connect to Ticket Generator's API for simple ticket/QR code generation, sending, and processing using Ticket Generator's Ticket Validator mobile app.

b) User Interfaces
i) The web app will provide the primary UI for customers & admin, optimized for all standard screen sizes.
ii) The web app will support basic accessibility features like screen reader compatibility.

c) Hardware Interfaces
i) The web app will run on modern devices and may use local storage.
ii) The system will interface with ticket validation scanning devices.

d) Software Interfaces
i) The system will interface with a database for consistent data storage.
ii) The system will interface with Stripe's API
iii) The system will interface with Ticket Generator's APIs

e) Communications Interfaces
i) All client-server communication occurs over HTTPS.

f) Memory
i) The web app will function within typical phone, tablet, and laptop memory limits and should not require continuous background operation.
ii) The backend will manage caching and database connections to support multiple users at the same time without exhausting server memory.

g) Operations
i) The backend services will run continuously (24/7), except during planned maintenance windows.
ii) The system will log errors and significant events to support debugging and maintenance.

h) Site Adaptation Requirements
i) The system will be built to accommodate different theater sizes, seat maps, discounts, and pricing.

2.2 Product Functions

- Users will be able to create an account using an email address, password, and optionally a phone number to purchase tickets.

- The system will allow users to buy at maximum 20 tickets at a time and reserve seats at theaters added to the system by theater company administrators.
- The system will be able to send end users a QR corresponding to their ticket bought to be used at their respective theater for verification.
- The system will allow for virtual method of payment and online purchasing of tickets by users
- The system will allow for theater company administrators to:
    - add and update movie screenings and time slots for the movie
    - room of each movie screening
    - ticket availability
    - cost of each ticket
- list available screenings at participating theaters added by movie theater company administrator
    - provide different options for ticket purchasing, such as discounts for children / seniors or theater promotions (for example, a loyalty program).
- System will prevent reselling of tickets

2.3 User Characteristics

Theater administrators:

1. Experienced in IT, software development, system management
2. Over the age of 18
3. Eligible to work in US

End Users (theater customers):

1. Generally teens - seniors
2. Has some familiarity with the use of technology such as making online purchases
3. Has an email

2.4 General Constraints

The MTTSS would be subject to the following general constraints that may limit the overall design and implementation options for developers.

1. Web Based Platform Constraint
    a. Must be implemented as a web application
    b. No native desktop or mobile application
2. Payment Processing Constraint
    a. All online payments must be processed through a third party payment service provider
    b. System must comply with payment service provider's usage policies and security requirements
3. Seat Availability
    a. Must prevent users from purchasing the same seat for the same movie screening

4. Time Constraint for Ticket Purchasing
    a. User must finish purchasing tickets within a fixed 10 minute window
        i. If payment is not processed within 10 minutes, reserved seats must appear available for other customers

2.5 Assumptions and Dependencies

These following assumptions and dependencies affect the requirements that are defined in this SRS document. If any changes were made, it may require to update the system requirements

It is assumed that:
1. Users have access to a modern web browser with a steady internet connection
2. Customers have a valid email address to receive the QR code based ticket
3. The theater staff will have access to a TC capable of scanning QR code for ticket verification
4. Users are willing to create an account before purchasing tickets online

The system depends on:
1. The third party payment service's API availability and reliability for payment processing
2. The ticket generator API for creating tickets, and checking whether the tickets are valid
3. Email and/or text of the tickets

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

3.1.1.1: User Interface #1 – Customer Web UI (Through Browser)
1. Name of item: Customer Web UI
2. Description of purpose:
    a. Provides the interface for customers to browse movies, select a showtime, choose seats, purchase tickets, and view their QR-code tickets.
3. Valid range, accuracy, and/or tolerance:
    a. Seat selection shall only allow seats marked as "available."
    b. Customers cannot select more than 20.
    c. Required fields (ex: email, payment step) shall be validated before checkout completes.
4. Timing:
    a. After a successful purchase, the confirmation page and ticket access shall be available within 5 minutes under normal operating conditions.

    b. If payment fails, the system shall display a failure message immediately (<1 second) after the payment attempt.
5. Relationships to other inputs/outputs:
    a. Retrieves movie listings, showtimes, pricing, and seat availability from the backend servers.
    b. Sends purchase requests to the backend, which communicates with the payment processor and ticket generator.
6. Window formats/organization:
    a. Runs in a standard browser tab/window and shall be responsive for desktop and mobile screen sizes.
7. Command formats:
    a. UI actions shall be available through standard web controls such as buttons and forms.
8. End messages:
    a. "Purchase Successful" / "Order Confirmed"
    b. "Payment Failed"
    c. "Seat No Longer Available"
    d. "Invalid Input" (missing/incorrect required fields)

3.1.1.2: User Interface #2 – Admin Web UI (Through Browser)

1. Name of item: Admin Web UI
2. Description of purpose:
    a. Provides the interface for admin to manage movies, configure theaters/rooms and seat maps, set ticket prices, and view sales/seat occupancy.
3. Valid range, accuracy, and/or tolerance:
    a. Admin shall not be able to create a screening with a showtime in the past.
    b. Required fields like: movie title, showtime, room selection, base ticket price shall be validated before saving changes.
    c. Pricing values shall not allow negative dollar amounts.
4. Timing:
    a. After saving an admin change like: add movie, update pricing, or create screening, the system shall confirm success/failure immediately (<2 seconds).
5. Relationships to other inputs/outputs:
    a. Sends management updates like: movies, rooms, seat maps, screenings, and pricing/discounts to the backend servers and database.
    b. Retrieves current system data from the backend servers.
    c. Updates what users see the customer UI.
6. Window formats/organization:
    a. Runs in a standard browser tab/window and shall be responsive for desktop and mobile screen sizes.
7. Command formats:
    a. UI actions shall be available through standard web controls such as buttons and forms.

8. End messages:
    a. “Saved Successfully”
    b. “Update Failed”
    c. “Validation Error” (missing/incorrect required fields)

3.1.2 Hardware Interfaces

3.1.2.1: Hardware Interface #1 – Theater Customer Devices (likely phones, tablets, or computers):

9. Name of item: Customer Device
10. Description of purpose:
    a. Provides the hardware platform customers will use to browse movies, select seats, purchase tickets, and view QR-code tickets.
11. Hardware must:
    a. Support modern web browsers like Chrome, Safari, and Microsoft edge.
    b. Have a modern device resolution.
12. Relationships to other inputs/outputs:
    a. Connects to web app via HTTPS.
    b. Users can open their tickets through their email.

3.1.2.2: Hardware Interface #2 – Theater Validation Devices (likely staff smartphones)

1. Name of item: Theater Validation Device (Smartphone)
2. Description of purpose:
    a. Provides the hardware for theater staff to use Ticket Generator’s ticket validation app to scan ticket QRcodes.
3. Device must:
    a. Support modern app stores like Apple Store or Google Play Store so that Ticket Generator’s ticket validation app can be downloaded.
4. Relationships to other inputs/outputs:
    a. Communicates with Ticket Generator’s backend via HTTPS.

3.1.2.3: Hardware Interface #3 – Admin/Staff Devices (likely computers)

1. Name of item: Admin/Staff Devices (computers)
2. Description of purpose:
    a. Provides the hardware platform for admin to manage/add movies, create seat maps, and set ticket prices.
3. Hardware must:
    a. Support modern web browsers like Chrome, Safari, and Microsoft Edge.
    b. Have a modern device resolution.
4. Relationships to other inputs/outputs:

    a. Connects to web app via HTTPS.
    b. Users can open their tickets through their email.

3.1.2.4: Hardware Interface #4 – Company Servers:

1. Name of item: Company Servers (Backend/Hosting Servers)
2. Description of purpose:
    a. Provides the hardware that hosts the theater ticketing system's backend services and database, allowing customers and admins to use the web app.
3. Hardware must:
    a. Support running the backend web server/application and database services.
    b. Support secure internet connectivity for handling HTTPS requests.
    c. Handle at least 1000 users accessing the system at the same time.
4. Relationships to other inputs/outputs:
    a. Receives requests from Customer Devices and Admin/Staff Devices over HTTPS.
    b. Sends responses back to those devices (movie listings, seat availability, purchase confirmations, validation results).
    c. Communicates with third-party services (payment processor Stripe and Ticket Generator) over HTTPS.

3.1.3 Software Interfaces

3.1.3.1: Software Interface #1 – Payment Processing (Stripe API)

1. Name of item: Stripe Payment Interface
2. Description of purpose:
    a. Provides payment processing for customer ticket purchases.
3. Software must:
    a. Send correct payment requests to Stripe to charge the customer accurately for the order total.
    b. Receive a success or failure result from Stripe before confirming an order.
    c. Payment requests sent to Stripe to be accurate 99.99% of the time.
4. Relationships to other inputs/outputs:
    a. Receives the final purchase amount from the checkout page in the Customer Web UI.
    b. Returns payment success/failure to the backend, which determines whether tickets are generated and the order is confirmed.
    c. If payment succeeds, the backend proceeds to send the info over to the Ticket Generator interface to create QR-code tickets and have them sent.
5. Data formats:
    a. Payment request/response data exchanged via JSON.

3.1.3.2: Software Interface #2 – Payment Processing (Stripe API)

6. Name of item: Stripe Payment Interface

7. Description of purpose:
    a. Provides payment processing for customer ticket purchases.
8. Software must:
    a. Send correct payment requests to Stripe to charge the customer accurately for the order total.
    b. Receive a success or failure result from Stripe before confirming an order.
    c. Payment requests sent to Stripe to be accurate 99.99% of the time.
9. Relationships to other inputs/outputs:
    a. Receives the final purchase amount from the checkout page in the Customer Web UI.
    b. Returns payment success/failure to the backend, which determines whether tickets are generated and the order is confirmed.
    c. If payment succeeds, the backend proceeds to send the info over to the Ticket Generator interface to create QR-code tickets and have them sent.
10. Data formats:
    a. Payment request/response data exchanged via JSON.

3.1.2.3: Software Interface #2 – Database Interface:

1. Name of item: Database
2. Purpose:
    a. Store user and admin account info, movie and seat map info, general theater info, and purchases.
3. Software must:
    a. Store and get customer account info correctly.
    b. Store and get movies, showtimes/screenings, theaters/rooms, and seat maps correctly.
    c. Store and get seat availability and reservations correctly.
    d. Maintain data consistency so that one seat cannot be sold to more than one person for the same movie.
4. Relationship to other inputs/outputs:
    a. Receives updates from the Admin Web UI about movies, screenings, seat maps, and pricing/discounts.
    b. Gives data to the Customer Web UI like: movie listings, showtimes, location, pricing, and seat availability.
    c. Works with the payment interface and ticket generator interface to finalize orders and keep track of ticket records after successful payments.
5. Data Formats:
    a. Records, queries, tables, collections

3.1.4 Communications Interfaces

3.1.4.1: Communication Interface #1 – Customer/Admin <–> Backend:

1. Protocol: HTTPS
2. Purpose:
    a. Browsing movies, seat availability, login, checkout, admin management
3. Data format:
    a. JSON requests/responses
4. Errors:
    a. consistent HTTP status codes (200/400/401/403/404/409/500)

3.1.4.2: Communication Interface #2 – Backend <–> Payment Processor (Stripe):

1. Protocol: HTTPS
2. Purpose:
    a. Process payments, confirm success/failure
3. Timing:
    a. Payment result returned during checkout (<3 seconds)
4. Example items:
    a. "Payment processing requests shall be sent to Stripe over HTTPS."
    b. "Payment events will be verified before updating order status."

3.1.4.3: Communication Protocol #2 – Backend <–> Ticket Generator API:

1. Protocol: HTTPS
2. Purpose:
    a. Generate ticket/QR after successful purchase and have it sent to customer
3. Data format:
    a. JSON post requests
4. Example items:
    a. "After successful payment, the backend shall request ticket generation from Ticket Generator over HTTPS."

3.2 Functional Requirements

This section describes specific features of the software project. If desired, some requirements may be specified in the use-case format and listed in the Use Cases Section.

3.2.1 – Feature 1: Ticket purchasing and handling

1. The system shall allow users to input their banking information to purchase tickets
2. The system shall send user data to Ticket Generator's API for ticket generation and emailing.
3. The system shall prevent the purchasing of tickets over the respective limit
4. The system shall prevent the purchase of an already bought ticket
5. The system shall update ticket availability if changes are made (example: refund)
6. The system shall notify and cancel any purchases with conflicting banking information
7. The system shall factor in discounts from promotions or certain age groups during purchase

8. The system shall prevent the purchasing of tickets if the user is unable to afford them

3.2.2 – Feature 2: Graphics / webapp

1. The system shall display all available movies
2. The system shall show any potential discounts / promotions set by the theater
3. The system shall be scalable and readable regardless of device
4. The system shall display the theaters in which a movie is showing
5. The system shall display the number of tickets in inventory as well as the available seats per screening

3.2.3 – Feature 3: Emailing System

1. The system shall notify users about purchases
2. The system shall email users about promotions / discounts going on
3. The system shall email users about errors that occurred during purchase or account creation

3.2.4 – Feature 4: Account Creation

1. The system shall allow users to create an account using an email
2. The system shall allow users to optionally add a phone number to an account
3. The system shall notify users about errors in account creation
4. The system shall allow users to update or add information after account creation

3.2.5 – Feature 5: Ticket Validation

1. The system shall validate and verify QR codes that are used for redeeming tickets at theaters
2. The system shall prevent the use of unauthorized QR codes to be used at theaters
3. The system shall ensure that ticket QR codes are not replicable

3.2.6 – Feature 6: Admin Account

1. The system shall allow the creation of special administrative accounts to be used by employees
2. The system shall allow admin accounts to create, update and delete movie listings on the website
3. The system shall allow admins to add certain promotions or discounts to the website

3.2.7 – Feature 7: Promotions

1. The system shall allow for theaters to add promotions to the website that can affect pricing of tickets.

3.3 Use Cases:

3.3.1 Use Case #1: Family Ticket Purchase

A father who is located inside San Diego is buying 5 tickets for his entire family. His family consists of two children, one teen, and two adults.

3.3.2 Use Case #2: Single Customer

A single person who wants to purchase one ticket for themselves at any FFMTC location. The person logs into their account , selects movie, showtime, seat, and goes to checkout.

3.3.3 Use Case #3: Trying to Purchase Reserved Seat

When someone tries to select a seat that is already taken, it will not select the seat, and will give the user a message saying that seat is unavailable for this screening.

3.3.4 Use Case #4: Update Movie Information

Admins are able to update moving listings, showtimes, prices, seat count, and overall theater availability.

3.4 Classes / Objects

3.4.1 – Class #1: UserAccount (Abstract)	3.4.2 – Class #2: Ticket	3.4.3 – Class #3: MovieListing
Attributes userID: String userName: String userPassword:String userEmail:String	Attributes availability: boolean ticketPrice: double ticketID: string movieTitle:String ticketNumber: int theaterLocations: Arr	Attributes movieName:String movieID:String ticketInventory: int theaterLocations: Arr ticketPrice: double movieDescription: String
Operations verifyLogin(String password): bool isPassword(String password): bool logIn(String userID): void signOut(String userID): void errorHandling(): void	Operations isAvailable(bool availability): bool errorHandling(): void	Operations seatAvailability(int ticketInventory): bool isAtLocation(): bool errorHandling(): void
References	References	References

Account Creation Emailing System	Ticket purchasing and handling Ticket validation	Ticket purchasing and handling Ticket validation Graphics / webapp

3.4.4 – Class #4: AdminUser extends UserAccount	3.4.5 – Class #5: Payment	3.4.6 – Class #6: Customer extends UserAccount
Attributes storeNum: int	Attributes userPayment: String userID: String userEmail: String userName: String ticketPrice: double billingAddress: String purchaseValidate: bool cardNum: String cardExprDate: String cardCVC: String	Attributes tickets: ArrayList<Ticket>
Operations createListing(string movieName, int ticketInventory, string movieID, Arr theaterLocations, double ticketPrice, String movieDescription): void errorHandling(): void updatePrice(MovieListing movie, string movieID, double price): void updateName(String movieID, string movieName): void	Operations isValid(): bool processPayment(): void updateTicketAvailability(): void refundPurchase(): void cancelPurchase(): void errorHandling(): void promotionHandling(): void	Operations void buyTicket(): void void refundTicket(): void

updateInventory(String movieID, int ticketInventory): void updateTheaterLocations(String movieID, String theater): void updateDescription(String movieID, String description): void deleteListing(String movieID): void addPromotion(): void		
References Account Creation Emailing System Graphics / webapp Admin Account Promotions	References Ticket purchasing and handling Ticket validation Promotions	References Account Creation Emailing System

3.5 Non-Functional Requirements

3.5.1 Performance

1. Roughly 90% of purchases should be completed within 2 seconds when at checkout, not including actual charge due to delays from the third party payment provider to the bank.
2. The MTTSSshould support many users purchasing tickets at once without slowing down the core functionality.

3.5.2 Reliability

1. The MTTSSshould have the correct ticket availability and seat assignments when users are on the system.
2. The MTTSSwill NOT allow duplicate tickets and seats being sold for the same movie screening

3.5.3 Availability

1. The MTTSSshall be available for users to purchase tickets 24/7, with the exception of scheduled maintenance.
2. Scheduled maintenance should be known to the movie theater administrators in advance.

3.5.4 Security

The MTTSSshall:

1. Securely store users' account information
2. Not store raw payment data (such as credit cards)
3. Encrypt communication between client and server
4. Restrict administrative functions to only authenticated administrators

3.5.5 Maintainability

The MTTSSshall:

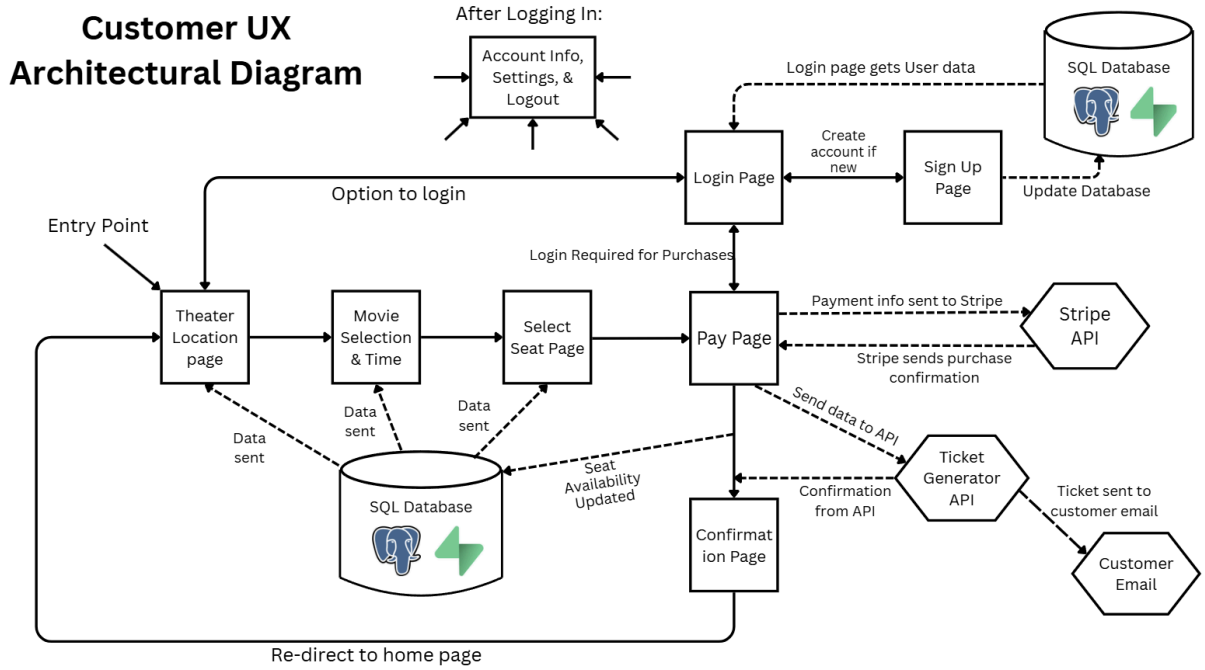
1. Allow administrators to update the movies, showtimes, pricing, and even promotions without needing to restart the MTTSSevery time
2. Log errors and any significant event to help with debugging

3.5.6 Portability

1. The MTTSSshall operate on 95% of modern web browsers

3.6 Customer Architectural Diagram – Updated to show DB choices

3.6.1 Diagram

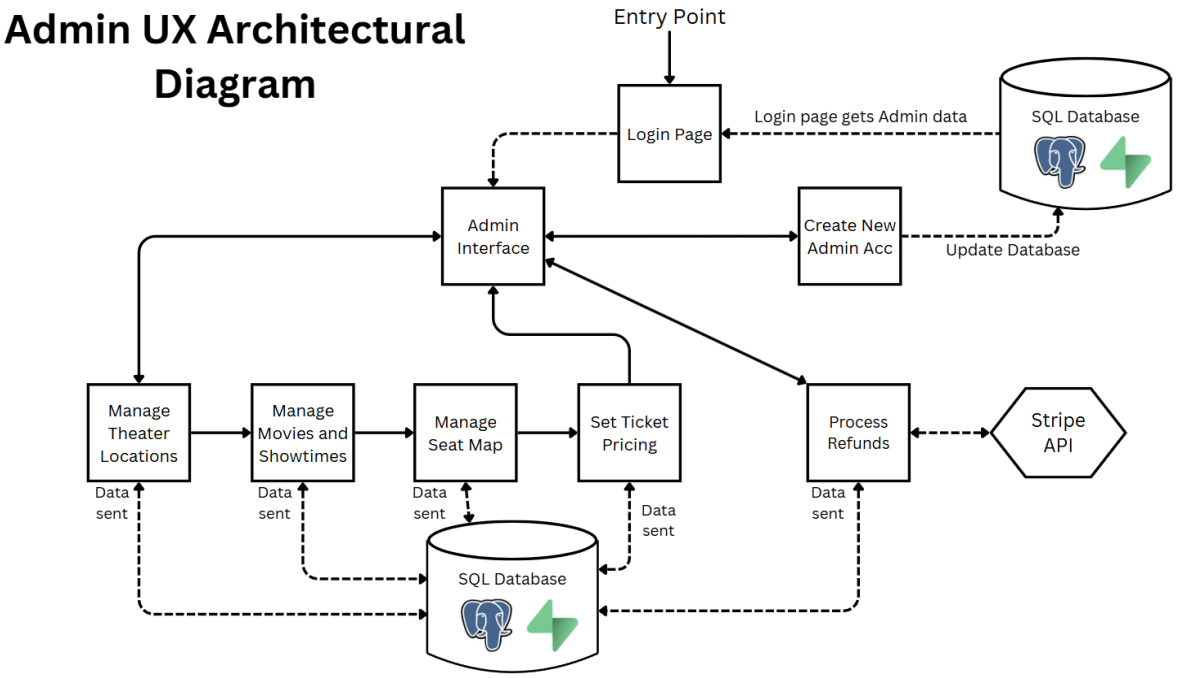


3.6.2 Description

The user/customer begins at the home page which lists all the different theater locations the customer can select from. This data is retrieved from the database containing a list of all the FFMTC's theater locations in San Diego. From there the customer has the choice to select a theater or login first. If the customer chooses to login, they will be taken to the login page. If their credentials are found in the user database, they will be sent back to the home page now logged in. If they do not already have an account, they will have the option to create one, updating the user database and taking them to the login page where they should now be able to login. After logging in, they will be taken to the home page where they can select a theater. After selecting a theater, the customer will be given the option to browse current movies playing at that theater on different dates and at different times. Once the customer has selected a date, a movie, and a time, they will be taken to the seat selection page (which lists the price of seats). After selecting their seats, the customer will be taken to the payment page where they will be forced to login or create a new account if they haven't yet done so. After that, they will be taken back to the payment page where they can enter their credit card information, which is then sent to Stripe's API. After the system receives the purchase confirmation from Stripe's API, it sends the customer's data and ticket info to Ticket Generator's API (which emails the users their tickets) and then takes the user to a purchase confirmation page. The system will also update the seat availability database to take into account the newly purchased seats. The purchase confirmation page will give the customer the option to return to the home page if they would like to keep browsing for movies, or they can simply exit the site.

3.7 Administrator Architectural Diagram – Updated to show DB choices

3.7.1 Diagram



3.7.2 Description

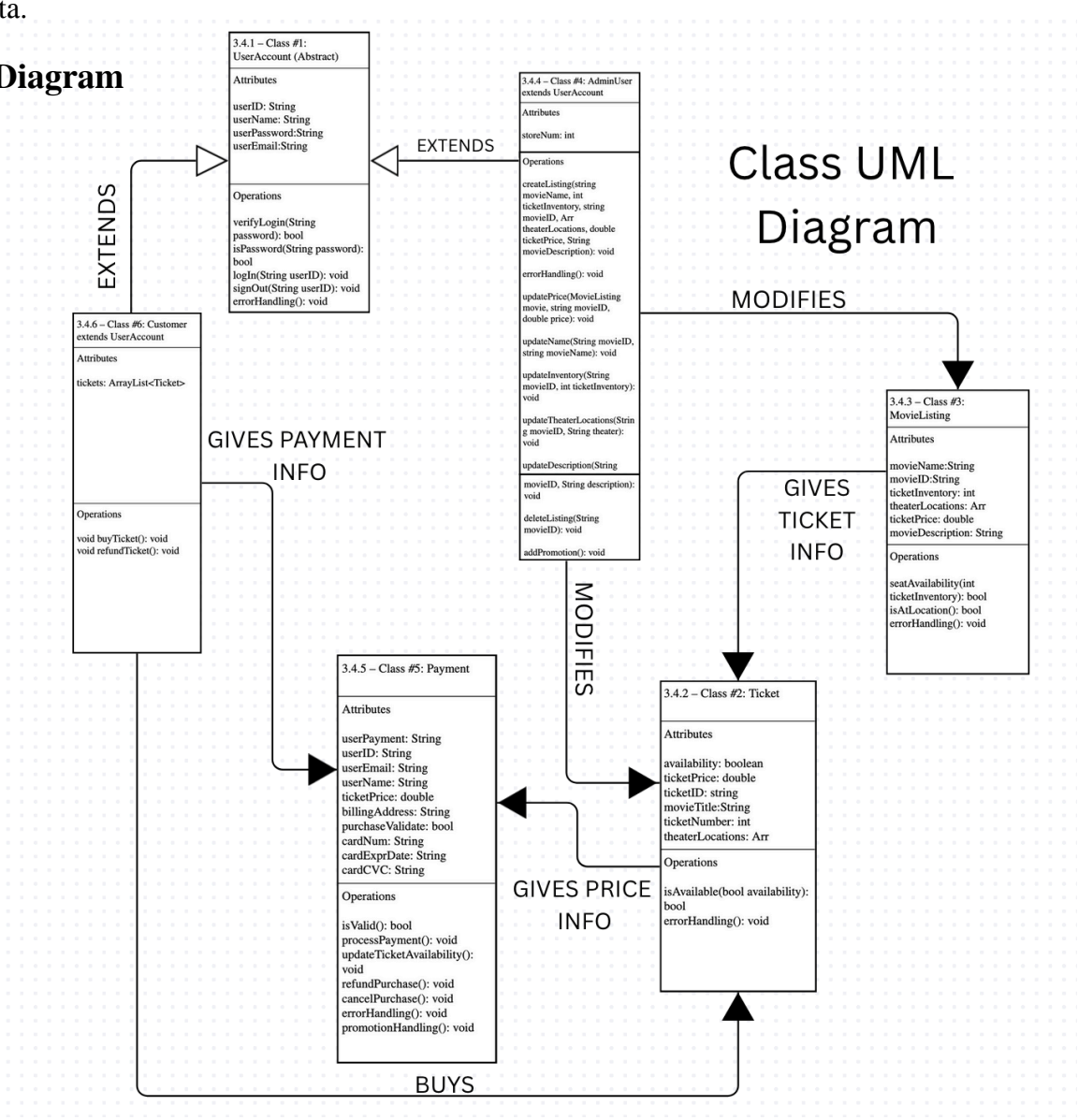
The administrator work flow starts when a theater admin opens the login page and enters their email and password. The system goes ahead and verifies the user's account by using the Admin Accounts database, which holds the data for each employee. If the login is successful, the employee is granted access to the admin interface page. From this page, the admin can go between pages that allow them to:

1. Create new admin accounts
2. Manage the theater locations
3. Manage movies and showtimes
4. Manage the seat map per auditorium
5. Set ticket prices
6. Process refunds

Each task is verified and/or updates the database that relates to said task. As for the process refunds task, it simply calls the Stripe API to then provide a customer with their refund, and updates the sales data.

3.8 UML Class Diagram

3.8.1 Diagram



KEY:

BLACK ARROW = ASSOCIATION

WHITE ARROW = INHERITANCE

3.8.2 Description

UserAccount is connected to Customer and AdminUser class. It acts as an abstract class for both, containing userID, userName, userPassword and userEmail. Its operations primarily handle logging in and out. AdminUser modifies the data contained by the ticket class and MoveListing class. Customer class stores the data for users that create accounts. It provides the payment info used in Payment class as well as buys tickets from the Ticket class. The Ticket class serves as a template for tickets that are bought by users. This class gets information from MovieListing which is then used by the Payment class during the payment process. The Payment class implements the functionality needed for a customer to buy and refund tickets. It is dependent on the Customer class for payment and the ticket class for the actual price of the purchase. MovieListing class is the main class for displaying and getting information from the theater's current showings. It provides the movie names, amount of tickets available, price, and description.

4. Development Plan

4.1 Timeline

Month	Collaboration	Eric	Leland	Matthew
January	Discussing which sections of the program each collaborator will be working on. **Leland:** - Frontend Development - Architecture Diagram **Eric:** - Class Development - UML Class Diagram **Matthew:** - Database Development - API Implementation	- Mapping out necessary classes the program requires - Specifying which classes will be for customer end and administrator end	- Mapping out necessary web pages the web application requires - Planning features and overall design each webpage will contain	- Start planning for database - Finding required APIs to utilize (Stripe, Ticket Generator)

February	- Review and complete SRS document - Define the system architecture, such as the frontend, backend, and database - Decide how the components will talk to each other	- Refine and complete UML class diagram - Define members and methods for each class	- Design UI for all main pages on webapp - Design a seat selection interface - Plan how UI will look on different devices such as mobile vs desktop	- Design a database scheme - Create a table for users, movies, screenings, seats, tickets, and purchase histories - Plan an API interaction flow with Stripe and Ticket Generator
March	- Finalize integration plan with the frontend, backend, and database	- Implement core classes required for MTTS - Create the logic for ticket purchasing, seat availability, and user accounts	- Create frontend prototype pages - Implement a way to navigate between the different pages - Design a seat map - Create a purchase workflow UI	- Implement the database scheme - Create backend API points for the movie listings, seat availability, and ticket purchasing - Begin the integration of Stripe API
April	- Combine frontend, backend and database - plan testing sessions - identify bugs and system issues	- Debug the required class logic - Ensure that the seat reservation logic prevents any duplicates from being purchased - Test ticket purchasing flow	- Improve the UI usability across devices - Add any needed error messages and validation - Optimize each page responsiveness	- Complete the Stripe API integration - Implement the Ticket Generator API - Verify the ticket generation and QR code are functioning
May	- Final system review - Ensure that each requirement in SRS is met - Prepare for final presentation and documentation	- Document class behaviors and how they work	- Final UI adjusting	- Final database check - Ensure data integrity and seat availability are consistent - Document API interactions

4.2 GITHUB LINKS
Eric Daniel's Repo
Leland's Repo
Matthew's Repo

5. Test Plan

5.1 SDS Test Plan

Our test cases mostly focused on stress testing each of the classes and their various functionality. For example, one main focus of our tests is the validation of ticket purchases and payment processes. These tests help us ensure the security of these various processes and ensure satisfactory delivery of tickets and potential refunds. We also tested out the ability of the server to create and save user accounts created, showing that integration of our database with our website was successful. Finally, we were able to successfully test the creation of Administrative accounts and their various commands, such as adding a new movie to the website's listing.

6. Data Management Strategy

6.1 Database Plan

For our project, our first decision was to use an SQL database as opposed to a non-SQL database. One of the reasons why is due to SQL's data integrity and security. SQL follows ACID protocol (Atomicity, Consistency, Isolation, and Durability), which provide guarantees for data validity. It ensures operations performed on the database are performed to completion or cancelled, and data operations follow the set parameters of data changes. In addition, operations can't contradict each other, so something like double booking seats is impossible. Finally, it is resistant even to system failure. We chose to go with a PostgreSQL database and have the hosting done by SupaBase. After considering the trade offs and what other databases and hosts offer such as MySQL and Neon, we decided to use these two because of their reliability, open source nature, and ability to handle complex data queries. Despite their complexity to learn and resource consumption, we believe that these are the best for handling the data of MTTSS. While MySQL could also work, our team decided against MySQL because of its potentially slower server response time and backup issues. For now, we only plan on using one primary Postgres database. We chose Supabase over Neon as Supabase automatically comes with several services like OAuth (account creation & authentication) and Supabase instantly provides REST and GraphQL endpoints for our tables.

PostgreSQL	MySQL

+Reliable and advanced	+Good at handling smaller datasets
+Open source	+Open source
+Great Scalabilty	+Ease of Use
+Monitoring tools for uptime and performance	+Cross platform compatible

6.2 SQL Tables Architecture Diagram

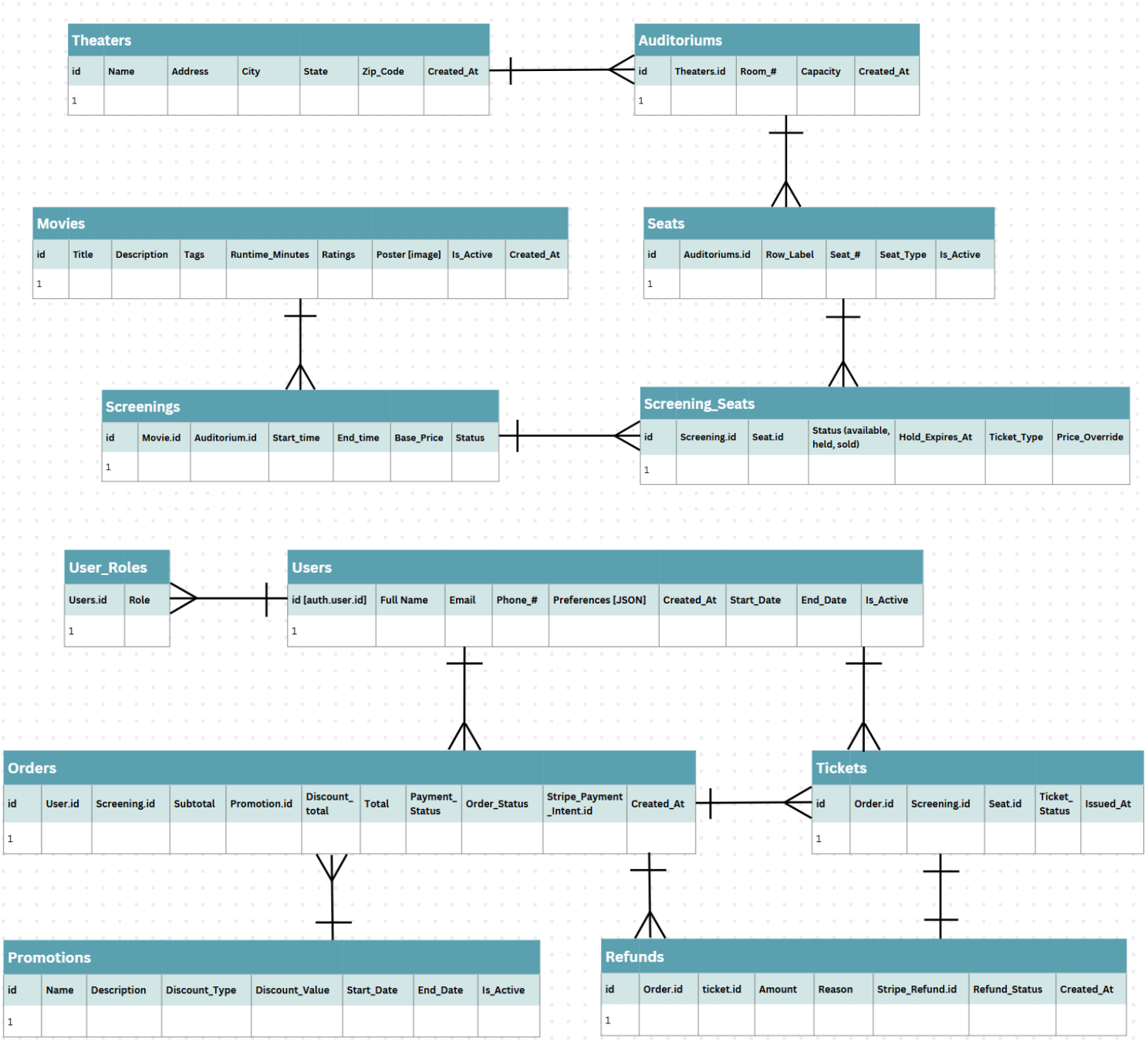


Diagram Description:

Each table in the SQL Table Architecture Diagram contains a column for ID, which is simply the key to access the data for that specific row of data. Also, some of the tables contain a column for Created_At, which is simply when each row was added to the table of the database.

For the Theaters' table, the columns are simply about the address of the location, containing things such as street address, city, zip code, etc. The Theaters' table has a one-to-many relationship with the Auditorium table. This is because there can be many auditoriums in a theater, but not many theaters that are part of one auditorium.

For the Auditoriums' table, there is a column for Theater ID to indicate which theater this auditorium belongs to. There are also columns for the room number assigned to the auditorium, and the number of seats in the auditorium. The Auditoriums' table has a one-to-many relationship with the Seats's table.

For the Seats' table, there is a column for the auditorium ID to indicate which auditorium the seat belongs to. There are also columns for the seat's row and seat number to describe the location of the seat within the auditorium. There is also a column for seat type, to specify if the seat is a standard seat, or a more comfortable seat. The Seats' table has a one-to-many relationship with the Screening-Seats table.

For the Screening_Seats' table, there is a column for Seat ID and Screening ID to indicate which seat and screening (will go more over later) the Screening_Seat belongs to. There is also a column for the status of the seat to showcase whether the seat is reserved, or available.

For the Movies' table, there are columns that describe what movie it is, its descriptions and tags, and the runtime described in minutes. There is also a column to describe whether the movie is currently being shown in theaters or not. The Movies' table has a one-to-many relationship with the Screenings' table.

For the Screenings' table, there is a column for the movie ID to indicate which movie will be showcased during the screening. There are also columns for the start and end time for the screening, and the base price a ticket would cost for this screening. The Screenings' table also has a one-to-many relationship with the Screening_Seats' table, just like the Seats' table.

For the Users' table, there are columns for the user's name, contact information, and the date their account was created, and when (if at all), their account is deleted. The Users' table has a one-to-many relationship with the User_Roles' table, the Orders' table, and even the Tickets' table.

For the User_Roles' table, there is only one column that describes the user's role, whether they are a customer, or an admin.

For the Promotions' table, there are columns that describe what kind of promotion it is, when the promotion starts, ends, and whether or not it is currently active. The Promotions' table has a one-to-many relationship with the Orders' table just like the Users' Table.

For the Orders' table, there are columns for the user ID to determine which user the order belongs to, and one for the screening ID to see which screening the order belongs to. There are also columns to hold the order's status and the total the user was charged. The Orders' table has a one-to-many relationship with the Tickets' table and the Refunds's table.

For the Tickets' table, there are columns for the seat ID, order ID, and screening ID to indicate which one the ticket belongs to. There are also columns for the ticket's status, and also one for the date the ticket was issued. The Tickets' table has a one-to-one relationship with the Refunds' table because the user should be able to refund per ticket instead of per order.

For the Refunds' table, there are columns for the order ID and the ticket ID to indicate which order and ticket the refund will belong to. There are also columns for the amount of money for the refund and the reason. There is also a Stripe API ID because we use Stripe API for our payment process.

*We also updated our old architecture diagrams to show our updated database choices, i.e., using PostgreSQL and SupaBase.

7. Change Management Process

Identify and describe the process that will be used to update the SRS, as needed, when project scope or requirements change. Who can submit changes and by what means, and how will these changes be approved.

A. Appendices

Appendices may be used to provide additional (and hopefully helpful) information. If present, the SRS should explicitly state whether the information contained within an appendix is to be considered as a part of the SRS's overall set of requirements.

Example Appendices could include (initial) conceptual documents for the software project, marketing materials, minutes of meetings with the customer(s), etc.

A.1 Appendix 1

A.2 Appendix 2